Module 37: Graded Mini Project

Student: Manoj Bhardwaj

Project: Conditional Generative Adversarial Network (cGAN) for Cats vs Dogs image generation.
The model conditions both generator and discriminator on class labels where 0 = Cat and 1 = Dog.

Project Objective

Build an end-to-end cGAN pipeline that loads the TensorFlow Datasets `cats_vs_dogs` dataset, preprocesses the images, defines conditional generator and discriminator networks, trains them adversarially for at least 10 epochs, and saves label-conditioned visual outputs.

Rubric Alignment

Criterion	Implementation Evidence
Dataset Loading	Uses `tfds.load('cats_vs_dogs')` with 90/10 train/test split and verifies 0 = Cat, 1 = Dog.
Preprocessing	Resizes to 64x64, normalizes to [-1, 1], casts labels to int32, batches and prefetches.
Generator	Noise and label inputs; label embedding; concatenation; Conv2DTranspose upsampling; tanh 64x64x3 output.
Discriminator	Image and label inputs; spatial label embedding; image-label concatenation; Conv2D real/fake logit.
Training	`@tf.function` train step with separate generator and discriminator gradient updates.
Loss and Optimizers	Binary cross-entropy from logits; separate Adam optimizers with learning rate 2e-4 and beta_1 0.5.
Visualization	Saves real sample grid, untrained baseline grid, and epoch-wise conditioned generated grids.
Interpretation	Notebook includes prompts for image quality, diversity, label conditioning, limitations, and improvements.

Execution Instructions

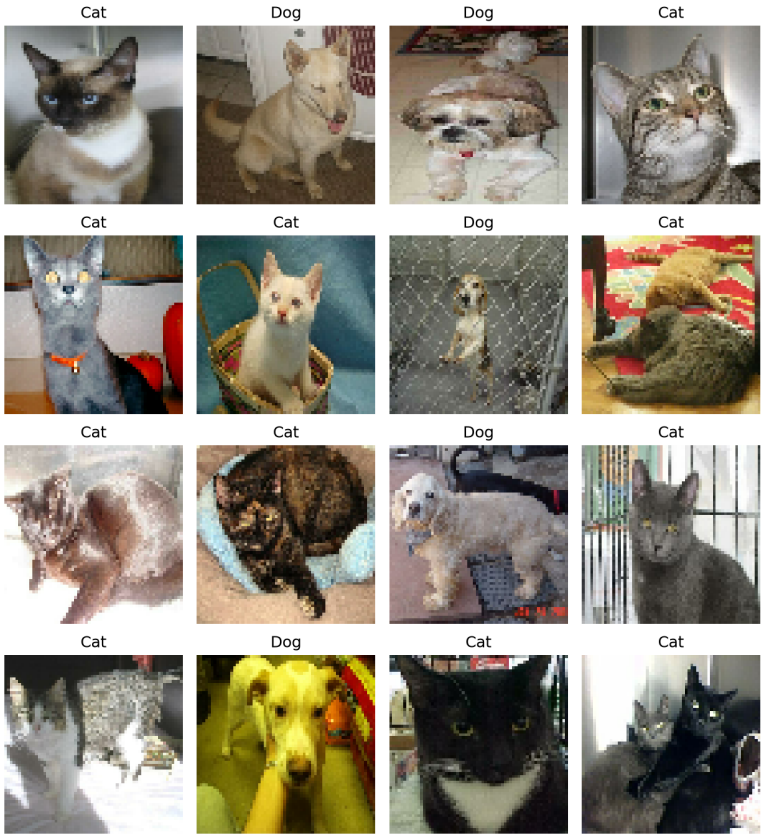
Recommended Windows runtime: Python 3.11. Create a virtual environment, install `requirements.txt`, then run `python src\cgan_cats_vs_dogs.py --epochs 10 --batch-size 128 --data-dir .tfds`.
If local certificate verification blocks the TFDS download, add `--disable-download-ssl-verification`.
For a fast no-download validation, run `python src\cgan_cats_vs_dogs.py --architecture-check`.
For a dataset pipeline check, run `python src\cgan_cats_vs_dogs.py --smoke-test`.

Output Files

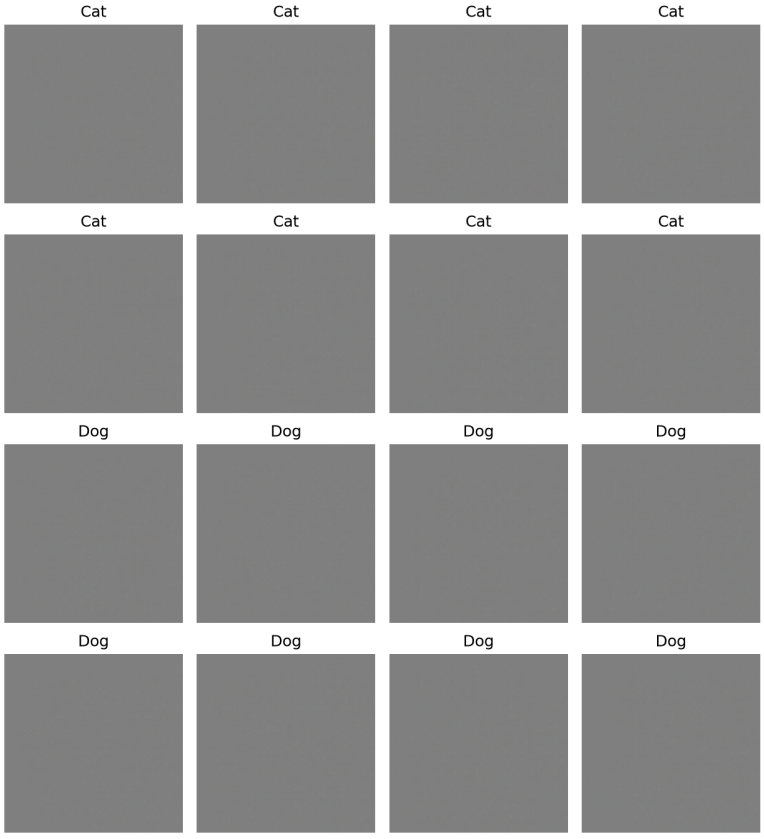
The implementation saves generated grids in `outputs/samples/`, including `real_preprocessed_samples.png`, `epoch_000_untrained.png`, and one generated image grid per epoch. The final trained models are saved as Keras files in `outputs/models/`.

Visual Output Samples

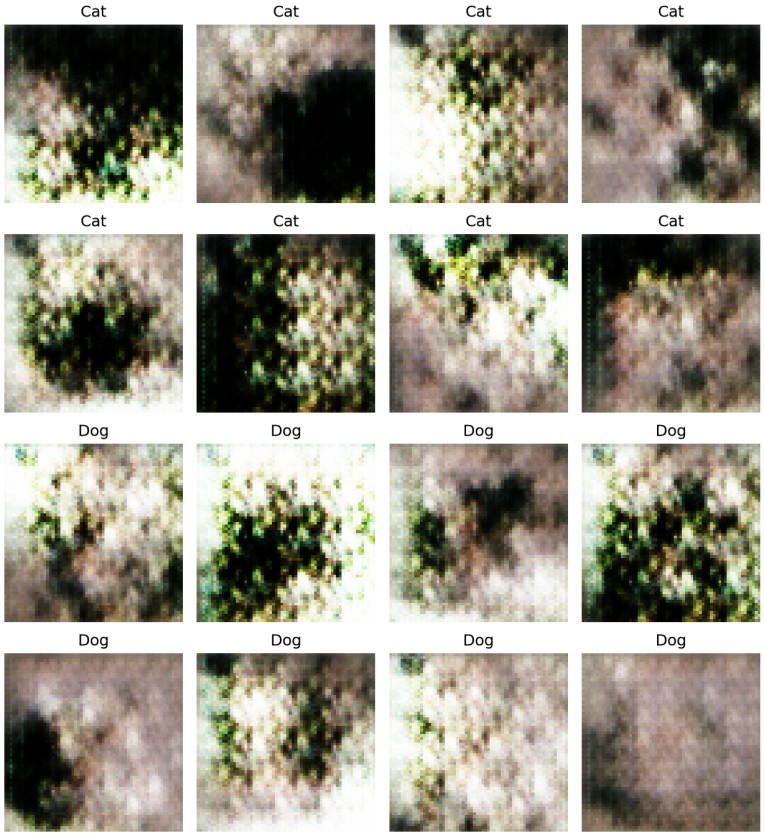
Real preprocessed validation samples



Untrained generator baseline



Latest class-conditioned generated grid (epoch_010.png)



Appendix: Complete Python Implementation

```
"""Conditional GAN for class-conditioned Cats vs Dogs image generation.

This module implements the complete Week 37 graded mini-project pipeline:

1. Load the Cats vs Dogs dataset with tensorflow_datasets.
2. Resize images to 64x64 and normalize pixels to [-1, 1].
3. Build a conditional generator and discriminator that both consume labels.
4. Train with binary cross-entropy and separate Adam optimizers.
5. Save class-conditioned image grids for evaluation.

Label convention required by the assignment:
    0 = Cat
    1 = Dog
"""

from __future__ import annotations

import argparse
import dataclasses
import io
import os
import time
import zipfile
from pathlib import Path
from typing import Any, Optional

# Keep the module importable for static inspection in environments where the
# TensorFlow stack is not installed yet. Runtime functions call
# ensure_dependencies() before using these modules.
try:
    import matplotlib.pyplot as plt
except ModuleNotFoundError:
    plt = None

try:
    import numpy as np
except ModuleNotFoundError:
    np = None

try:
    import tensorflow as tf
except ModuleNotFoundError:
    tf = None

try:
    import tensorflow_datasets as tfds
except ModuleNotFoundError:
    tfds = None


def tf_function_when_available(function: Any) -> Any:
    """Apply tf.function when TensorFlow is installed; otherwise leave as-is."""

    if tf is None:
        return function
    return tf.function(function)


@dataclasses.dataclass(frozen=True)
class CGANConfig:
    """Central configuration for the cGAN experiment."""

    image_size: int = 64
    channels: int = 3
    num_classes: int = 2
    latent_dim: int = 100
    batch_size: int = 128
    shuffle_buffer: int = 10000
    epochs: int = 10
    learning_rate: float = 2e-4
    beta_1: float = 0.5
    seed: int = 42
```

```
    sample_grid_rows: int = 4
    sample_grid_cols: int = 4
    output_dir: Path = Path("outputs")


def ensure_dependencies() -> None:
    """Raise a clear error if the TensorFlow stack is not available."""

    missing = []
    if tf is None:
        missing.append("tensorflow")
    if tfds is None:
        missing.append("tensorflow-datasets")
    if plt is None:
        missing.append("matplotlib")
    if np is None:
        missing.append("numpy")

    if missing:
        packages = " ".join(missing)
        raise ModuleNotFoundError(
            "Missing required runtime packages: "
            f"{packages}. Install them with: pip install -r requirements.txt"
        )


def patch_tfds_cats_vs_dogs_windows_paths() -> None:
    """Patch a TFDS Cats vs Dogs ZIP path issue seen on Windows.

    TFDS 4.9.x normalizes ZIP member names with os.path.normpath inside the
    Cats vs Dogs builder. On Windows this changes `PetImages/Cat/0.jpg` into
    `PetImages\\Cat\\0.jpg`, which can break when the recoded image is written
    back to an in-memory ZIP archive. The assignment requires TFDS, so this
    patch keeps using the TFDS builder while preserving ZIP-style `/` paths.
    """

    ensure_dependencies()
    if os.name != "nt":
        return

    from tensorflow_datasets.image_classification import cats_vs_dogs as cats_module

    if getattr(cats_module.CatsVsDogs, "_windows_zip_path_patch_applied", False):
        return

    def _generate_examples(self: Any, archive: Any) -> Any:
        num_skipped = 0
        for fname, fobj in archive:
            norm_fname = fname.replace("\\", "/")
            res = cats_module._NAME_RE.match(norm_fname)
            if not res:
                continue

            label = res.group(1).lower()
            if tf.compat.as_bytes("JFIF") not in fobj.peek(10):
                num_skipped += 1
                continue

            img_data = fobj.read()
            img_tensor = tf.image.decode_image(img_data)
            img_recoded = tf.io.encode_jpeg(img_tensor)

            buffer = io.BytesIO()
            with zipfile.ZipFile(buffer, "w") as new_zip:
                new_zip.writestr(norm_fname, img_recoded.numpy())
            buffer.seek(0)
            zip_reader = zipfile.ZipFile(buffer)
            new_fobj = zip_reader.open(norm_fname)

            record = {
                "image": new_fobj,
                "image/filename": norm_fname,
```

```
                "label": label,
            }
            yield norm_fname, record

        if num_skipped != cats_module._NUM_CORRUPT_IMAGES:
            raise ValueError(
                f"Expected {cats_module._NUM_CORRUPT_IMAGES} corrupt images, "
                f"but found {num_skipped}."
            )
        cats_module.logging.warning(
            "%d images were corrupted and were skipped",
            num_skipped,
        )

    cats_module.CatsVsDogs._generate_examples = _generate_examples
    cats_module.CatsVsDogs._windows_zip_path_patch_applied = True


def label_names_from_info(ds_info: Any) -> list[str]:
    """Return TFDS label names and validate the assignment's label convention."""

    ensure_dependencies()
    names = list(ds_info.features["label"].names)
    if names[:2] != ["cat", "dog"]:
        raise ValueError(
            "Unexpected label mapping from TFDS. Expected label 0 = cat and "
            f"label 1 = dog, but received: {names}"
        )
    return ["Cat", "Dog"]


def load_raw_datasets(
    data_dir: Optional[str] = None,
    verify_ssl: bool = True,
) -> tuple[Any, Any, Any]:
    """Load Cats vs Dogs using the exact TFDS split requested in the project."""

    ensure_dependencies()
    patch_tfds_cats_vs_dogs_windows_paths()
    download_config = tfds.download.DownloadConfig(verify_ssl=verify_ssl)
    (ds_train, ds_test), ds_info = tfds.load(
        "cats_vs_dogs",
        split=["train[:90%]", "train[90%:]"],
        shuffle_files=True,
        as_supervised=True,
        with_info=True,
        data_dir=data_dir,
        download_and_prepare_kwargs={"download_config": download_config},
    )
    return ds_train, ds_test, ds_info


def preprocess_image_label(image: Any, label: Any, image_size: int = 64) -> tuple[Any, Any]:
    """Resize image, normalize to [-1, 1], and cast labels to integer format."""

    ensure_dependencies()
    image = tf.image.resize(image, [image_size, image_size])
    image = tf.cast(image, tf.float32)
    image = (image / 127.5) - 1.0
    label = tf.cast(label, tf.int32)
    return image, label


def prepare_dataset(dataset: Any, config: CGANConfig, training: bool = True) -> Any:
    """Build an efficient tf.data pipeline with map, shuffle, batch, and prefetch."""

    ensure_dependencies()
    dataset = dataset.map(
        lambda image, label: preprocess_image_label(image, label, config.image_size),
        num_parallel_calls=tf.data.AUTOTUNE,
    )
    if training:
```

```
        dataset = dataset.shuffle(config.shuffle_buffer, seed=config.seed)
    dataset = dataset.batch(config.batch_size, drop_remainder=training)
    dataset = dataset.prefetch(tf.data.AUTOTUNE)
    return dataset


def denormalize_images(images: Any) -> Any:
    """Convert images from [-1, 1] back to [0, 1] for Matplotlib display."""

    ensure_dependencies()
    return tf.clip_by_value((images + 1.0) / 2.0, 0.0, 1.0)


def build_generator(config: CGANConfig) -> Any:
    """Build the conditional generator.

    Inputs:
        noise: random latent vector of shape (latent_dim,)
        class_label: integer label where 0 = cat and 1 = dog

    Output:
        64x64x3 RGB image with tanh activation, matching [-1, 1] preprocessing.
    """

    ensure_dependencies()
    layers = tf.keras.layers

    noise_input = tf.keras.Input(shape=(config.latent_dim,), name="noise")
    label_input = tf.keras.Input(shape=(), dtype=tf.int32, name="class_label")

    label_embedding = layers.Embedding(
        input_dim=config.num_classes,
        output_dim=config.latent_dim,
        name="label_embedding",
    )(label_input)
    label_embedding = layers.Flatten(name="flatten_label_embedding")(label_embedding)

    x = layers.Concatenate(name="noise_label_concatenate")(
        [noise_input, label_embedding]
    )
    x = layers.Dense(4 * 4 * 512, use_bias=False, name="project_to_feature_map")(x)
    x = layers.BatchNormalization(name="project_batch_norm")(x)
    x = layers.LeakyReLU(negative_slope=0.2, name="project_leaky_relu")(x)
    x = layers.Reshape((4, 4, 512), name="reshape_to_4x4")(x)

    for filters, name in [
        (256, "upsample_to_8x8"),
        (128, "upsample_to_16x16"),
        (64, "upsample_to_32x32"),
    ]:
        x = layers.Conv2DTranspose(
            filters,
            kernel_size=5,
            strides=2,
            padding="same",
            use_bias=False,
            name=f"{name}_conv_transpose",
        )(x)
        x = layers.BatchNormalization(name=f"{name}_batch_norm")(x)
        x = layers.LeakyReLU(negative_slope=0.2, name=f"{name}_leaky_relu")(x)

    image_output = layers.Conv2DTranspose(
        config.channels,
        kernel_size=5,
        strides=2,
        padding="same",
        activation="tanh",
        name="generated_rgb_image",
    )(x)

    return tf.keras.Model(
        inputs=[noise_input, label_input],
```

```
        outputs=image_output,
        name="conditional_generator",
    )


def build_discriminator(config: CGANConfig) -> Any:
    """Build the conditional discriminator.

    The class label is embedded into a spatial 64x64x1 conditioning map and
    concatenated with the RGB image before Conv2D feature extraction.
    """

    ensure_dependencies()
    layers = tf.keras.layers

    image_input = tf.keras.Input(
        shape=(config.image_size, config.image_size, config.channels),
        name="image",
    )
    label_input = tf.keras.Input(shape=(), dtype=tf.int32, name="class_label")

    label_embedding = layers.Embedding(
        input_dim=config.num_classes,
        output_dim=config.image_size * config.image_size,
        name="spatial_label_embedding",
    )(label_input)
    label_map = layers.Reshape(
        (config.image_size, config.image_size, 1),
        name="label_conditioning_map",
    )(label_embedding)

    x = layers.Concatenate(axis=-1, name="image_label_concatenate")(
        [image_input, label_map]
    )

    for filters, name in [
        (64, "downsample_to_32x32"),
        (128, "downsample_to_16x16"),
        (256, "downsample_to_8x8"),
        (512, "downsample_to_4x4"),
    ]:
        x = layers.Conv2D(
            filters,
            kernel_size=5,
            strides=2,
            padding="same",
            name=f"{name}_conv",
        )(x)
        x = layers.LeakyReLU(negative_slope=0.2, name=f"{name}_leaky_relu")(x)
        x = layers.Dropout(0.3, name=f"{name}_dropout")(x)

    x = layers.Flatten(name="flatten_features")(x)
    logits = layers.Dense(1, name="real_fake_logit")(x)

    return tf.keras.Model(
        inputs=[image_input, label_input],
        outputs=logits,
        name="conditional_discriminator",
    )


def generator_loss(cross_entropy: Any, fake_logits: Any) -> Any:
    """Generator loss: encourage fake images to be classified as real."""

    ensure_dependencies()
    return cross_entropy(tf.ones_like(fake_logits), fake_logits)


def discriminator_loss(cross_entropy: Any, real_logits: Any, fake_logits: Any) -> Any:
    """Discriminator loss: classify real images as real and generated images as fake."""

    ensure_dependencies()
```

```
    real_loss = cross_entropy(tf.ones_like(real_logits), real_logits)
    fake_loss = cross_entropy(tf.zeros_like(fake_logits), fake_logits)
    return real_loss + fake_loss


class CGANTrainer:
    """Owns optimizers and the optimized cGAN training step."""

    def __init__(self, generator: Any, discriminator: Any, config: CGANConfig):
        ensure_dependencies()
        self.generator = generator
        self.discriminator = discriminator
        self.config = config
        self.cross_entropy = tf.keras.losses.BinaryCrossentropy(from_logits=True)
        self.generator_optimizer = tf.keras.optimizers.Adam(
            learning_rate=config.learning_rate,
            beta_1=config.beta_1,
        )
        self.discriminator_optimizer = tf.keras.optimizers.Adam(
            learning_rate=config.learning_rate,
            beta_1=config.beta_1,
        )

    @tf_function_when_available
    def train_step(self, real_images: Any, class_labels: Any) -> tuple[Any, Any]:
        """Perform one alternating generator/discriminator update."""

        batch_size = tf.shape(real_images)[0]
        noise = tf.random.normal([batch_size, self.config.latent_dim])

        with tf.GradientTape() as gen_tape, tf.GradientTape() as disc_tape:
            generated_images = self.generator([noise, class_labels], training=True)

            real_logits = self.discriminator(
                [real_images, class_labels],
                training=True,
            )
            fake_logits = self.discriminator(
                [generated_images, class_labels],
                training=True,
            )

            gen_loss = generator_loss(self.cross_entropy, fake_logits)
            disc_loss = discriminator_loss(
                self.cross_entropy,
                real_logits,
                fake_logits,
            )

        generator_gradients = gen_tape.gradient(
            gen_loss,
            self.generator.trainable_variables,
        )
        discriminator_gradients = disc_tape.gradient(
            disc_loss,
            self.discriminator.trainable_variables,
        )

        self.generator_optimizer.apply_gradients(
            zip(generator_gradients, self.generator.trainable_variables)
        )
        self.discriminator_optimizer.apply_gradients(
            zip(discriminator_gradients, self.discriminator.trainable_variables)
        )

        return gen_loss, disc_loss


def make_fixed_sample_inputs(config: CGANConfig) -> tuple[Any, Any]:
    """Create fixed noise and labels for consistent epoch-to-epoch comparison."""

    ensure_dependencies()
```

```
    num_samples = config.sample_grid_rows * config.sample_grid_cols
    half = num_samples // 2
    labels = np.array([0] * half + [1] * (num_samples - half), dtype=np.int32)
    noise = tf.random.normal([num_samples, config.latent_dim])
    return noise, tf.constant(labels, dtype=tf.int32)


def save_generated_grid(
    generator: Any,
    noise: Any,
    labels: Any,
    class_names: list[str],
    output_path: Path,
    config: CGANConfig,
) -> None:
    """Generate class-conditioned images and save a labeled image grid."""

    ensure_dependencies()
    output_path.parent.mkdir(parents=True, exist_ok=True)

    generated_images = generator([noise, labels], training=False)
    display_images = denormalize_images(generated_images).numpy()
    label_values = labels.numpy().tolist()

    plt.figure(figsize=(config.sample_grid_cols * 2.1, config.sample_grid_rows * 2.3))
    for index, image in enumerate(display_images):
        plt.subplot(config.sample_grid_rows, config.sample_grid_cols, index + 1)
        plt.imshow(image)
        plt.title(class_names[label_values[index]])
        plt.axis("off")
    plt.tight_layout()
    plt.savefig(output_path, dpi=150, bbox_inches="tight")
    plt.close()


def save_real_sample_grid(
    dataset: Any,
    class_names: list[str],
    output_path: Path,
    config: CGANConfig,
    count: int = 16,
) -> None:
    """Save a quick grid of real preprocessed samples for validation."""

    ensure_dependencies()
    output_path.parent.mkdir(parents=True, exist_ok=True)

    for images, labels in dataset.take(1):
        display_images = denormalize_images(images[:count]).numpy()
        label_values = labels[:count].numpy().tolist()
        break
    else:
        raise ValueError("Dataset is empty; cannot save real sample grid.")

    rows = 4
    cols = 4
    plt.figure(figsize=(cols * 2.1, rows * 2.3))
    for index, image in enumerate(display_images[: rows * cols]):
        plt.subplot(rows, cols, index + 1)
        plt.imshow(image)
        plt.title(class_names[label_values[index]])
        plt.axis("off")
    plt.tight_layout()
    plt.savefig(output_path, dpi=150, bbox_inches="tight")
    plt.close()


def train_cgan(
    config: CGANConfig,
    data_dir: Optional[str] = None,
    verify_ssl: bool = True,
    max_train_batches: Optional[int] = None,
```

```
    sample_every: int = 1,
) -> tuple[list[dict[str, float]], Any, Any]:
    """Run the complete cGAN training workflow."""

    ensure_dependencies()
    tf.keras.utils.set_random_seed(config.seed)

    ds_train_raw, ds_test_raw, ds_info = load_raw_datasets(
        data_dir=data_dir,
        verify_ssl=verify_ssl,
    )
    class_names = label_names_from_info(ds_info)
    print(f"TFDS labels verified: 0 = {class_names[0]}, 1 = {class_names[1]}")

    train_dataset = prepare_dataset(ds_train_raw, config, training=True)
    test_dataset = prepare_dataset(ds_test_raw, config, training=False)

    output_dir = Path(config.output_dir)
    sample_dir = output_dir / "samples"
    model_dir = output_dir / "models"
    sample_dir.mkdir(parents=True, exist_ok=True)
    model_dir.mkdir(parents=True, exist_ok=True)

    save_real_sample_grid(
        test_dataset,
        class_names,
        sample_dir / "real_preprocessed_samples.png",
        config,
    )

    generator = build_generator(config)
    discriminator = build_discriminator(config)
    trainer = CGANTrainer(generator, discriminator, config)

    print("\nGenerator summary")
    generator.summary()
    print("\nDiscriminator summary")
    discriminator.summary()

    fixed_noise, fixed_labels = make_fixed_sample_inputs(config)
    save_generated_grid(
        generator,
        fixed_noise,
        fixed_labels,
        class_names,
        sample_dir / "epoch_000_untrained.png",
        config,
    )

    history: list[dict[str, float]] = []
    for epoch in range(1, config.epochs + 1):
        start_time = time.time()
        gen_loss_values = []
        disc_loss_values = []

        for step, (image_batch, label_batch) in enumerate(train_dataset):
            if max_train_batches is not None and step >= max_train_batches:
                break
            gen_loss, disc_loss = trainer.train_step(image_batch, label_batch)
            gen_loss_values.append(float(gen_loss.numpy()))
            disc_loss_values.append(float(disc_loss.numpy()))

        if not gen_loss_values:
            raise ValueError("No batches were processed during training.")

        epoch_record = {
            "epoch": float(epoch),
            "generator_loss": float(np.mean(gen_loss_values)),
            "discriminator_loss": float(np.mean(disc_loss_values)),
            "seconds": float(time.time() - start_time),
        }
        history.append(epoch_record)
```

```
        print(
            "Epoch {epoch:03d}/{total:03d} | "
            "gen_loss={gen:.4f} | disc_loss={disc:.4f} | seconds={seconds:.1f}".format(
                epoch=epoch,
                total=config.epochs,
                gen=epoch_record["generator_loss"],
                disc=epoch_record["discriminator_loss"],
                seconds=epoch_record["seconds"],
            )
        )

        if epoch % sample_every == 0:
            save_generated_grid(
                generator,
                fixed_noise,
                fixed_labels,
                class_names,
                sample_dir / f"epoch_{epoch:03d}.png",
                config,
            )

    generator.save(model_dir / "conditional_generator.keras")
    discriminator.save(model_dir / "conditional_discriminator.keras")

    return history, generator, discriminator


def run_architecture_check(config: CGANConfig) -> None:
    """Validate model shapes and one synthetic train step without TFDS download."""

    ensure_dependencies()
    tf.keras.utils.set_random_seed(config.seed)

    generator = build_generator(config)
    discriminator = build_discriminator(config)
    trainer = CGANTrainer(generator, discriminator, config)

    batch_size = 4
    labels = tf.constant([0, 1, 0, 1], dtype=tf.int32)
    noise = tf.random.normal([batch_size, config.latent_dim])
    generated_images = generator([noise, labels], training=False)
    fake_logits = discriminator([generated_images, labels], training=False)

    expected_image_shape = (
        batch_size,
        config.image_size,
        config.image_size,
        config.channels,
    )
    if tuple(generated_images.shape) != expected_image_shape:
        raise ValueError(
            f"Unexpected generator output shape: {generated_images.shape}; "
            f"expected {expected_image_shape}"
        )
    if tuple(fake_logits.shape) != (batch_size, 1):
        raise ValueError(
            f"Unexpected discriminator output shape: {fake_logits.shape}; "
            f"expected {(batch_size, 1)}"
        )

    real_images = tf.random.uniform(
        [batch_size, config.image_size, config.image_size, config.channels],
        minval=-1.0,
        maxval=1.0,
    )
    gen_loss, disc_loss = trainer.train_step(real_images, labels)
    print(f"Generator output shape: {generated_images.shape}")
    print(f"Discriminator output shape: {fake_logits.shape}")
    print(f"Synthetic train step losses: gen={gen_loss.numpy():.4f}, disc={disc_loss.numpy():.4f}")
```

```
def parse_args() -> argparse.Namespace:
    """Parse command-line arguments for the training script."""

    parser = argparse.ArgumentParser(
        description="Train a conditional GAN on TFDS Cats vs Dogs.",
    )
    parser.add_argument("--epochs", type=int, default=10)
    parser.add_argument("--batch-size", type=int, default=128)
    parser.add_argument("--latent-dim", type=int, default=100)
    parser.add_argument("--learning-rate", type=float, default=2e-4)
    parser.add_argument("--beta-1", type=float, default=0.5)
    parser.add_argument("--seed", type=int, default=42)
    parser.add_argument("--output-dir", type=Path, default=Path("outputs"))
    parser.add_argument("--data-dir", type=str, default=None)
    parser.add_argument("--sample-every", type=int, default=1)
    parser.add_argument(
        "--max-train-batches",
        type=int,
        default=None,
        help="Optional cap for quick development runs.",
    )
    parser.add_argument(
        "--smoke-test",
        action="store_true",
        help="Run one epoch and one batch to verify the pipeline quickly.",
    )
    parser.add_argument(
        "--architecture-check",
        action="store_true",
        help="Verify model shapes and one synthetic train step without downloading TFDS.",
    )
    parser.add_argument(
        "--disable-download-ssl-verification",
        action="store_true",
        help=(
            "Disable SSL certificate verification only for TFDS dataset download. "
            "Use this only when local certificate configuration blocks the download."
        ),
    )
    return parser.parse_args()


def main() -> None:
    """Entry point for script execution."""

    args = parse_args()
    epochs = 1 if args.smoke_test else args.epochs
    max_train_batches = 1 if args.smoke_test else args.max_train_batches

    config = CGANConfig(
        epochs=epochs,
        batch_size=args.batch_size,
        latent_dim=args.latent_dim,
        learning_rate=args.learning_rate,
        beta_1=args.beta_1,
        seed=args.seed,
        output_dir=args.output_dir,
    )
    try:
        if args.architecture_check:
            run_architecture_check(config)
        else:
            train_cgan(
                config,
                data_dir=args.data_dir,
                verify_ssl=not args.disable_download_ssl_verification,
                max_train_batches=max_train_batches,
                sample_every=args.sample_every,
            )
    except ModuleNotFoundError as exc:
        raise SystemExit(f"ERROR: {exc}") from None
```

```
if __name__ == "__main__":
    main()
```